Stack & Queue

assadullahkhan

AssadUllahKhan

Stack

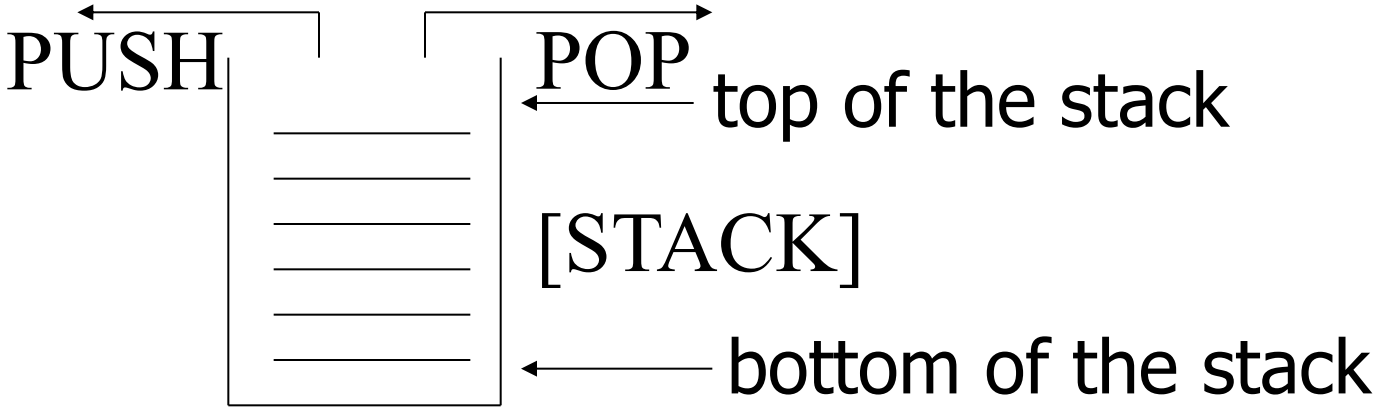
- A stack is also an ordered collection of elements like arrays, but it has a special feature that deletion and insertion of elements can be done only from one end called the top of the stack (TOP)
- Due to this property it is also called as last in first out type of data structure (LIFO).

Stack

- It could be through of just like a stack of plates placed on table in a party, a guest always takes off a fresh plate from the top and the new plates are placed on to the stack at the top.
- It is a non-primitive data structure.
- When an element is inserted into a stack or removed from the stack, its base remains fixed where the top of stack changes.

Stack

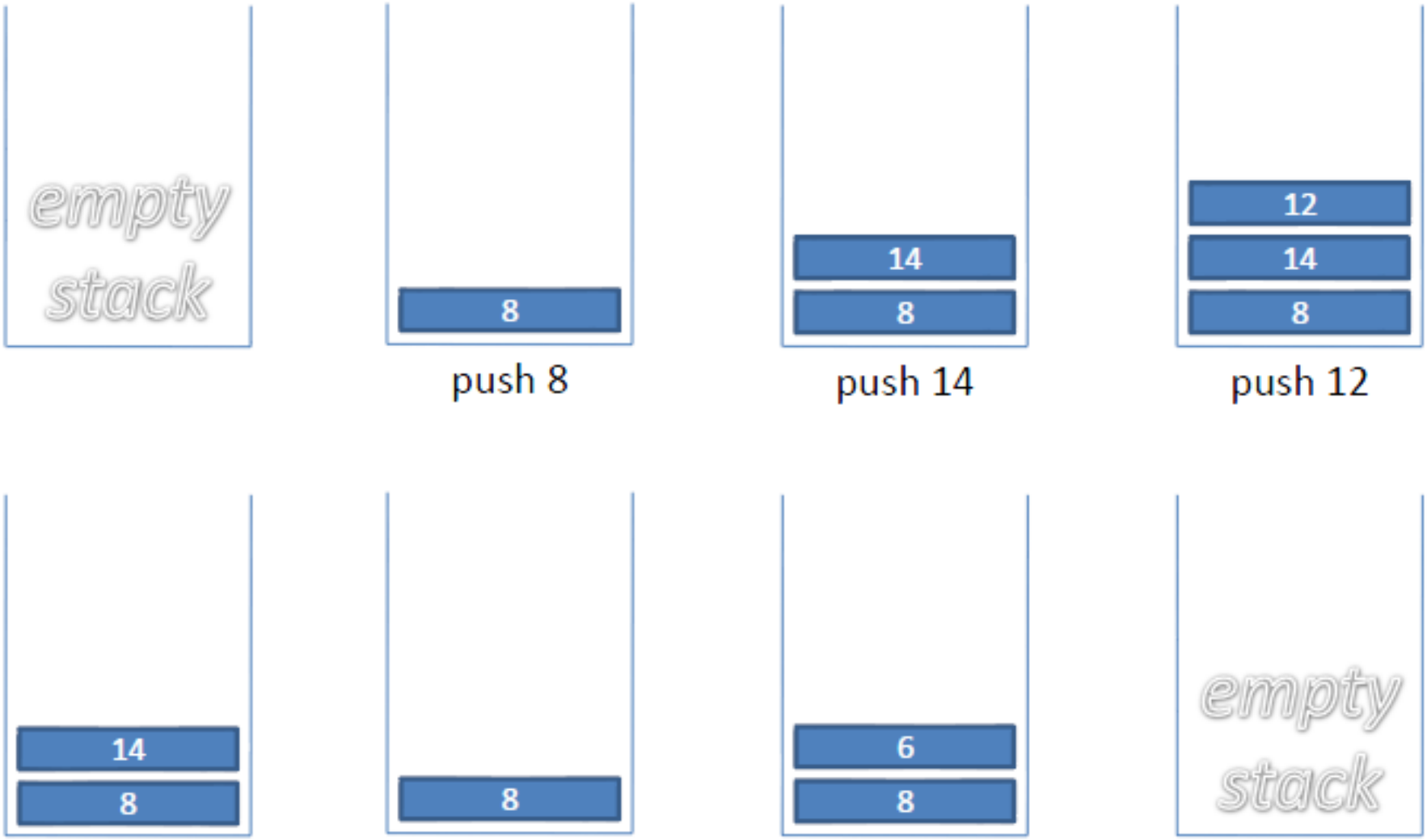
- Insertion of element into stack is called PUSH and deletion of element from stack is called POP.
- The bellow show figure how the operations take place on a stack:



Stacks

- Stack
  - New nodes can be added and removed only at the top
  - Similar to a pile of dishes
  - Last-in, first-out (LIFO)
  - Bottom of stack indicated by a link member to **NULL**
  - Constrained version of a linked list
- **push**
  - Adds a new node to the top of the stack
- **pop**
  - Removes a node from the top
  - Removes the value in reverse order
  - Returns **true** if **pop** was successful

Stack Example



Stack

- The stack can be implemented into two ways:
  - Using arrays (Static implementation)
  - Using pointer (Dynamic implementation)

Stack: Array or List

- Since both implementations support stack operations in constant time, any reason to choose one over the other?
- Allocating and deallocating memory for list nodes does take more time than preallocated array.
- List uses only as much memory as required by the nodes; array requires allocation ahead of time.
- List pointers (head, next) require extra memory.
- Array has an upper limit; List is limited by dynamic memory allocation.

Stack Abstract Data Type

- **Stack( )** creates a new stack that is empty.
- **push(item)** adds a new item to the top of the stack.
- **pop( )** removes the top item from the stack.
- **top( )** returns the top item from the stack but does not remove it.
- **isEmpty( )** tests to see whether the stack is empty.
- **isFull( )** tests to see whether the stack is Full.
- **size( )** returns the number of items on the stack.

Steps in Push Operation

1. Set the stack pointer to top
2. Check the stack for overflow (full), if full push operation can’t be performed
3. Increment the Top by 1.
4. If stack is not full insert the new data item

Algorithm for Push Operation

This algorithm will insert an element (item) on Stack (S) and N=Size of Stack or No. of memory locations and Top is the Top most location of the stack

Step 1: Push (S, Top, item) //*Creating a function*

Step 2: if (Top>=N) then //*Check for Overflow*

write("Overflow") & exit;

Step 3: Top=Top+1; //*Increment Top*

Step 4: S[Top]=newitem(value) //*Insert Value*

Step 5: Exit //*Finish*

Steps in Pop Operation

1. Check the stack for underflow (empty), if empty pop operation can't be performed and if empty then exit
2. Otherwise remove the value from top
3. Decrement the top by 1.

Algorithm for POP Operation

This algorithm will remove an element from Stack (S) and N=Size of Stack or No. of memory locations, Top is the Top most location of the stack and Temp is a Temporary variable

Step 1: Pop (S, Top) //*Creating a function*

Step 2: if (Top<1) then //*Check for Underflow*

write("Underflow") & exit;

Step 3: S[Top]=temp //*Delete the item from top*

Step 4: Top=Top-1; //*Decrement Top*

Step 5: Exit //*Finish*

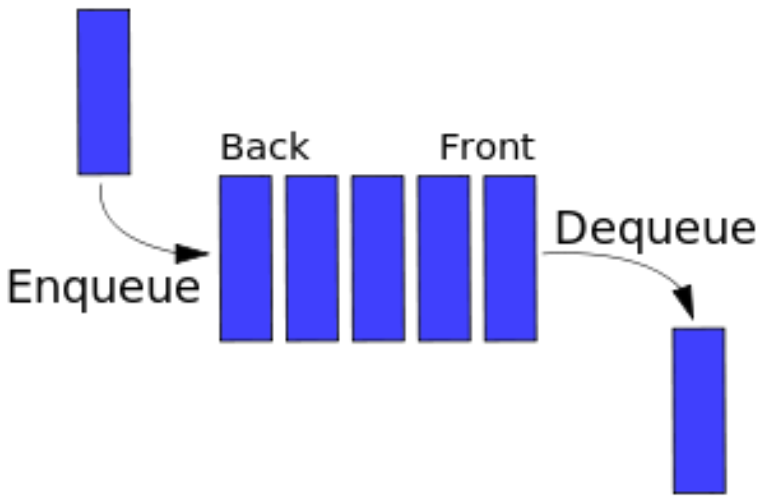
Implementation of Stack in C++

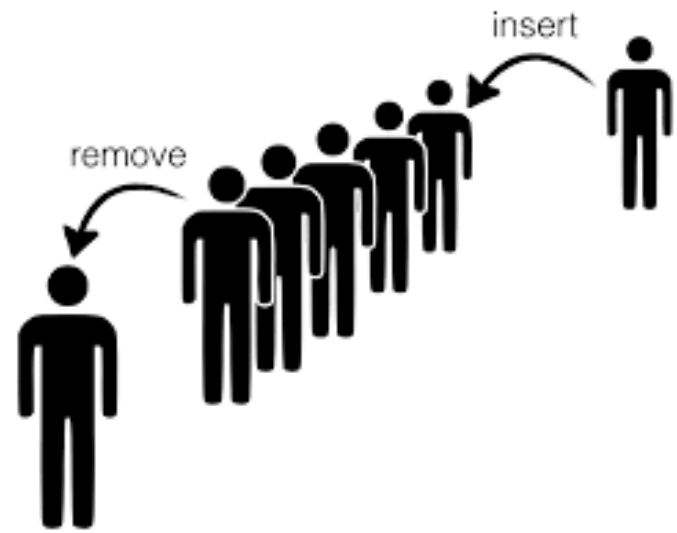
```
#include<iostream.h>
#include<conio.h>
void push( );
void pop( );
void display( );
void isFull( );
void isEmpty( );
int S[5], top=-1;
void main( )
{
  int choice;
  clrscr( );
   cout<<"Enter your choice \n 1. Push \n
              2. Pop  3. Display "<<endl;
    cin>>choice;
    switch(choice)
{  case 1: push( ); break;
case 2: pop( ); break;
 case 3: display( );  break;
 default:   cout<<"Wrong Value";
}
getch( );
}
void push( )
{
 isFull ( );
top=top+1;
cin>>S[top];
}
void isFull( )
{
 if (top= =4)
 cout<<"\n Stack is Full";
}
 void pop( )
{
 isEmpty( );
 S[top] = temp;
 top= top-1;
}
void isEmpty( )
{
 if (top = = -1)
 cout<<"Stack is empty"<<endl;
}
void display( )
{
for (int i=0; i<=top; i++)
cout<<S[i];
}
```

Queue

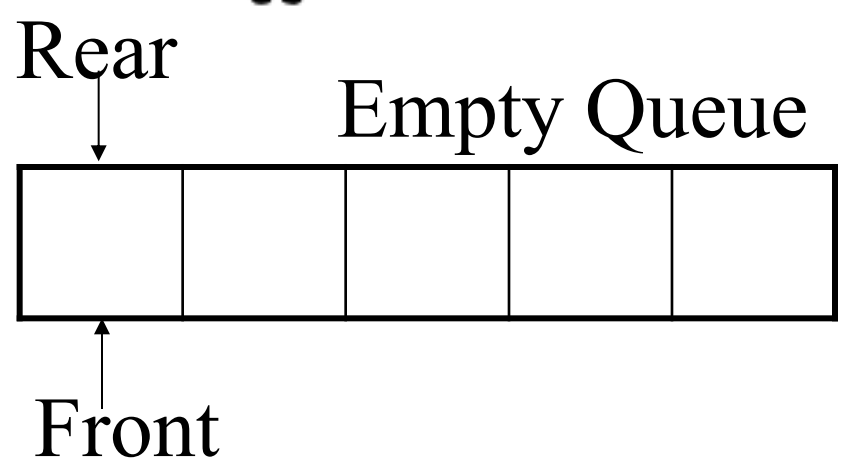
- A stack is LIFO (Last-In First Out) structure.
- In contrast, a queue is a FIFO (**F**irst-**I**n **F**irst-**O**ut) linear data structure.
- In a queue new elements are added to the queue from one end called REAR end and the element are always removed from other end called the FRONT end.
- The people standing in a railway reservation row are an example of queue.

Queue


Back
Front
Enqueue
Dequeue

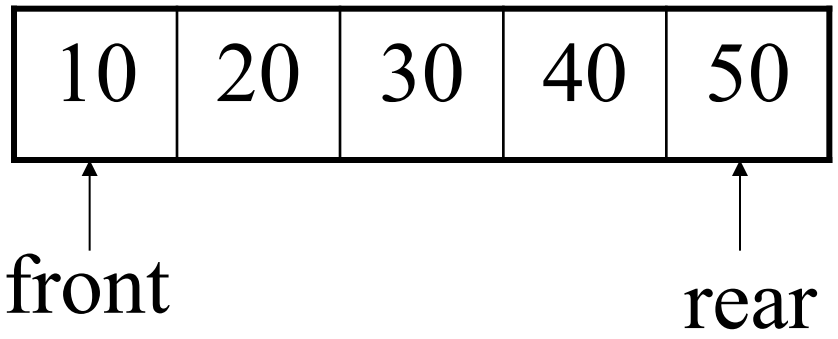

insert
remove

Queue is Full
25
30
51
60
85
45
88
90
75
95
front
rear


Rear
Empty Queue
Front

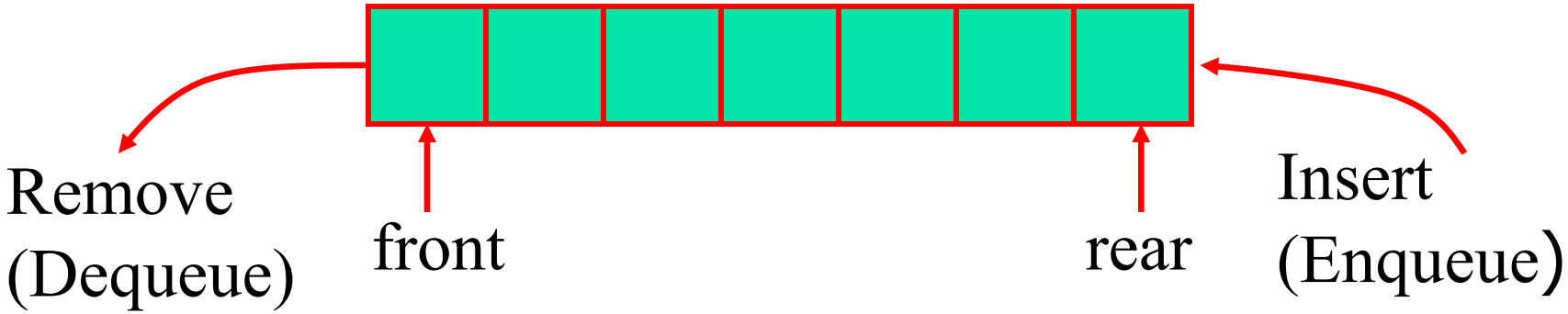
Queue

- Each new person comes and stands at the end of the row and person getting their reservation confirmed get out of the row from the front end.
- The below show figure how the operations take place on a stack:



Enqueue and Dequeue

- Primary queue operations: Enqueue and Dequeue
- Enqueue
  - Insert an element at the rear of the queue
- Dequeue
  - Remove an element from the front of the queue

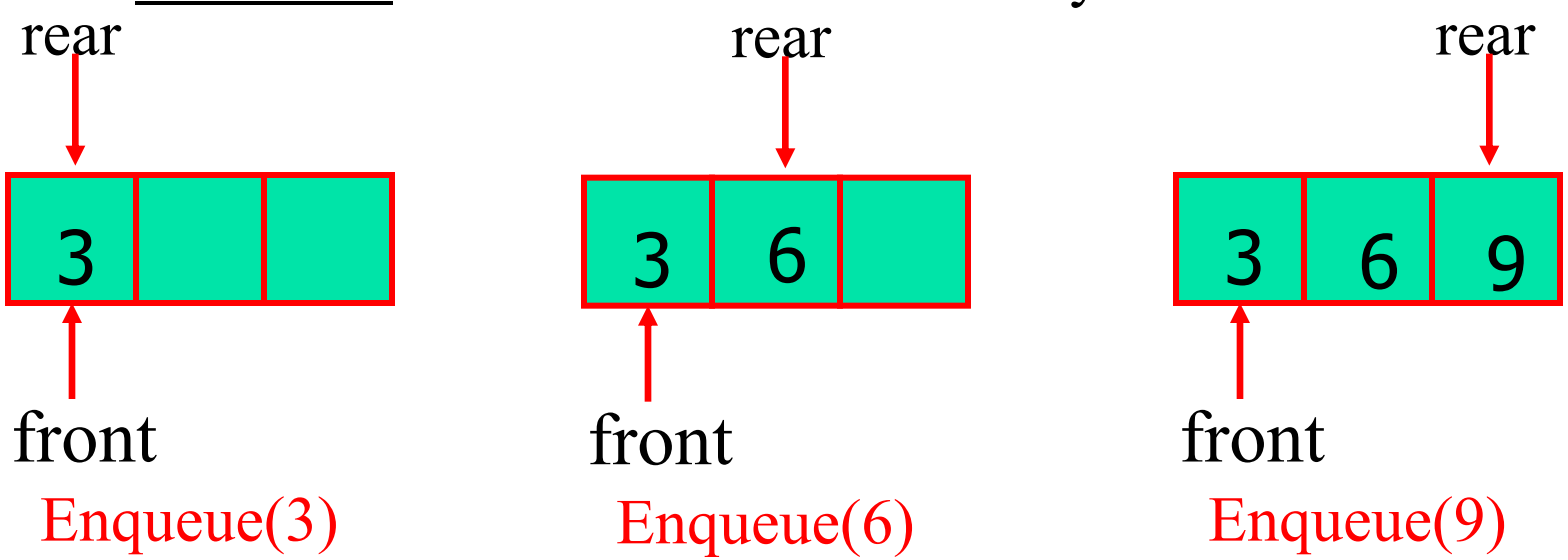


Queue

- The queue can be implemented into two ways:
  - Using arrays (Static implementation)
  - Using pointer (Dynamic implementation)

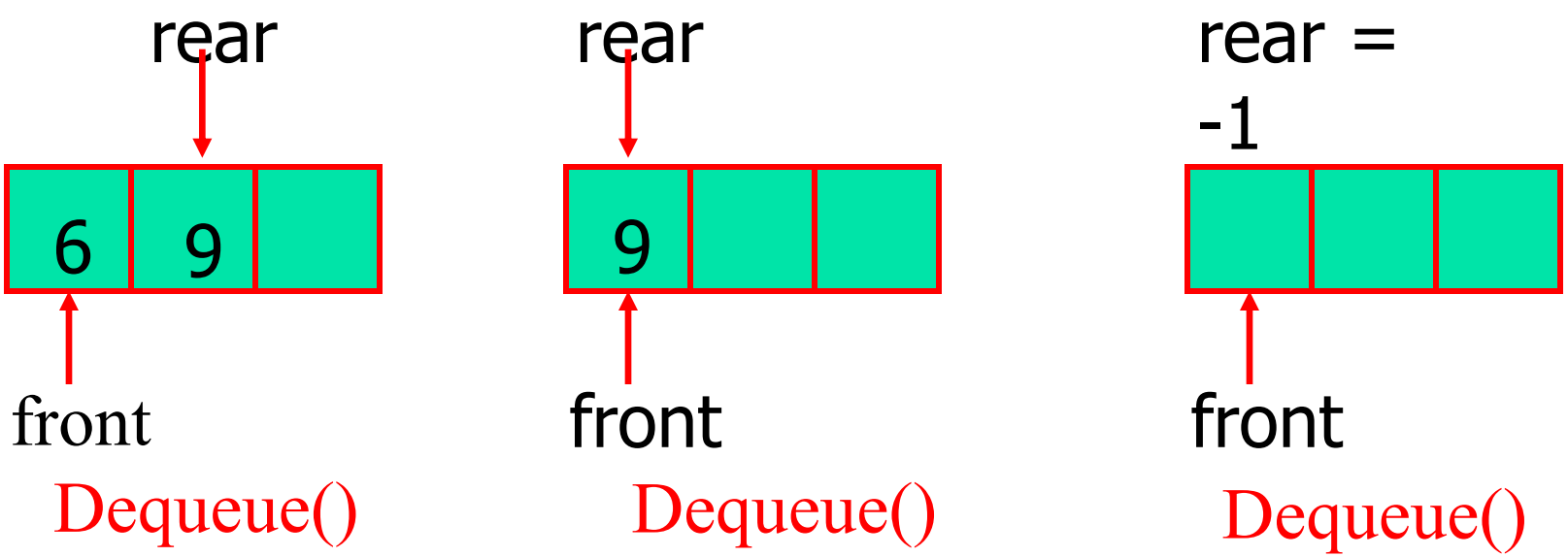
Queue Implementation of Array

- There are several different algorithms to implement Enqueue and Dequeue
- Naïve way
  - When enqueuing, the front index is always fixed and the rear index moves forward in the array.



Queue Implementation of Array

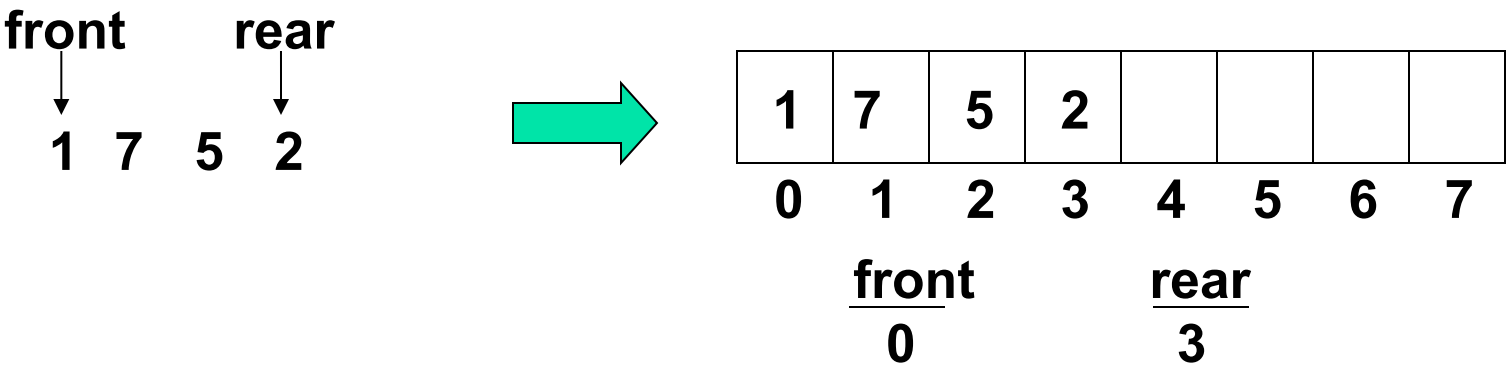
- Naïve way
  - When dequeuing, the element at the front the queue is removed. Move all the elements after it by one position. (Inefficient!!!)



Queue using Array

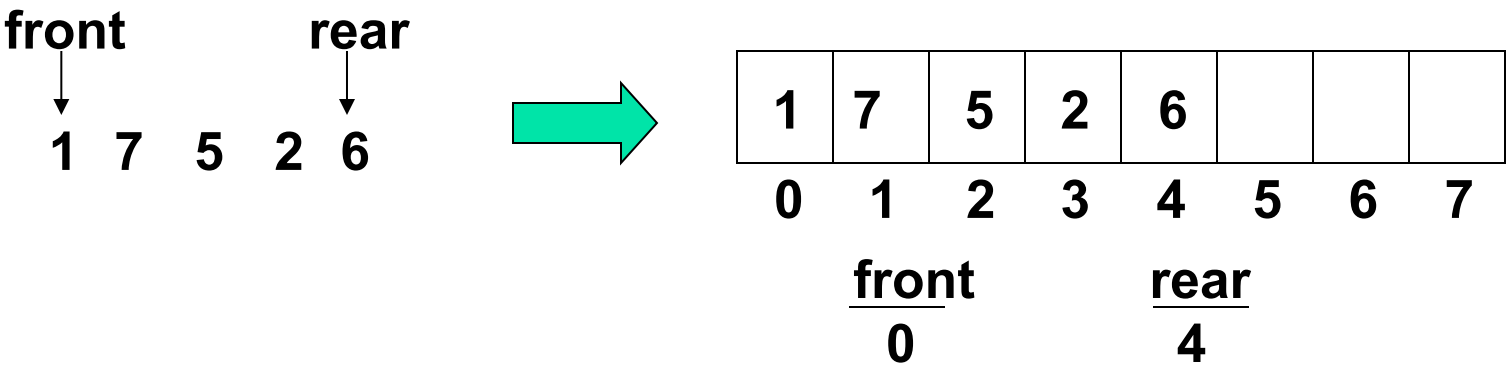
- If we use an array to hold queue elements, both insertions and removal at the front (start) of the array are expensive.
- This is because we may have to shift up to “n” elements.
- For the stack, we needed only one end; for queue we need both.
- To get around this, we will not shift upon removal of an element.

Queue using Array



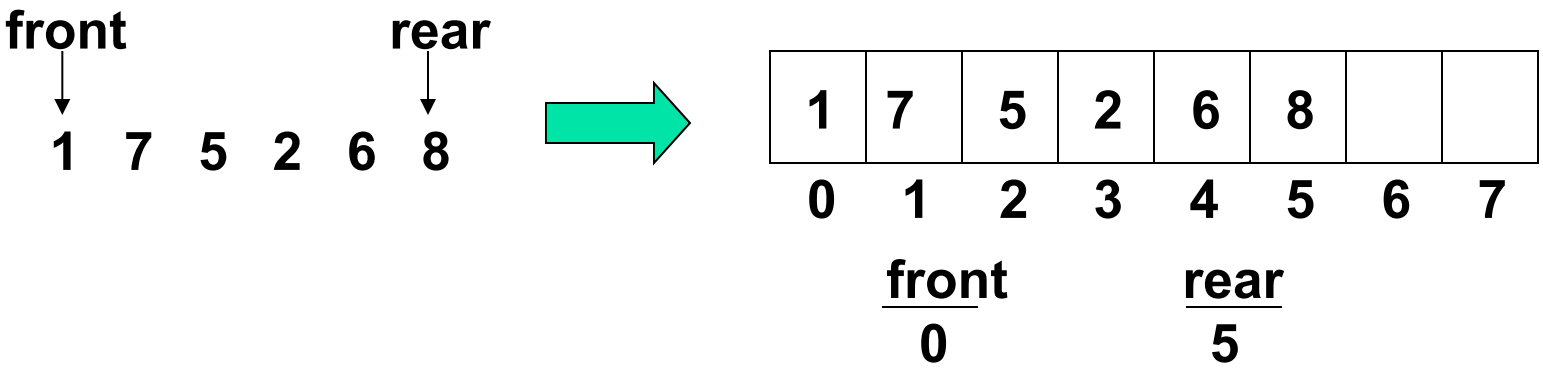
Queue using Array

enqueue(6)



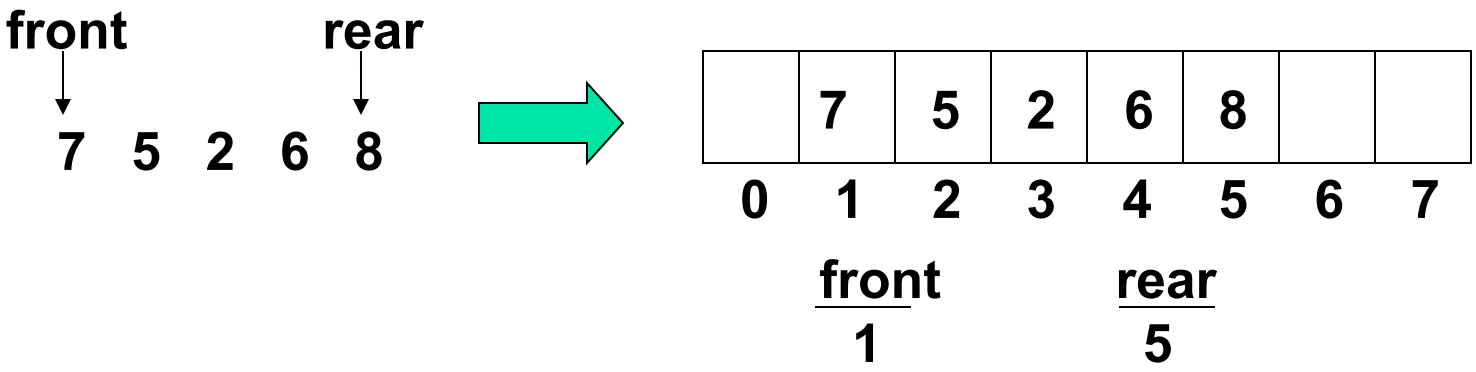
Queue using Array

enqueue(8)



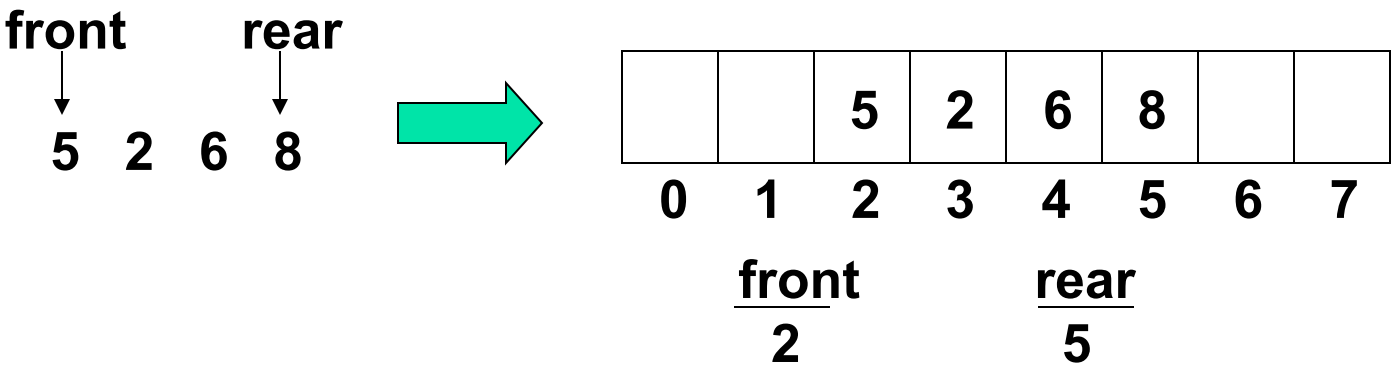
Queue using Array

dequeue()



Queue using Array

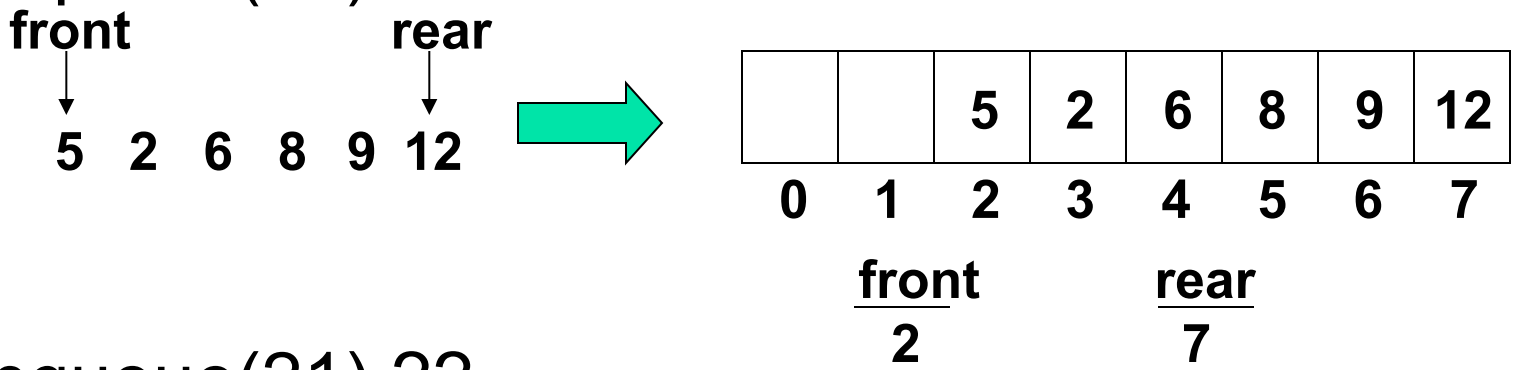
dequeue()



Queue using Array

enqueue(9)

enqueue(12)



enqueue(21) ??

Queue using Array

- We have inserts and removal running in constant time but we created a new problem.
- Cannot insert new elements even though there are two places available at the start of the array.
- Solution: allow the queue to “wrap around”.

Queue Implementation of Array

- Better way
  - When an item is enqueued, make the rear index move forward.
  - When an item is dequeued, the front index moves by one element towards the back of the queue (thus removing the front item, so no copying to neighboring elements is needed).

(front) XXXXOOOOO (rear)
OXXXXOOOO (after 1 dequeue, and 1 enqueue)
OOXXXXXOO (after another dequeue, and 2 enqueues)
OOOOXXXXX (after 2 more dequeues, and 2 enqueues)

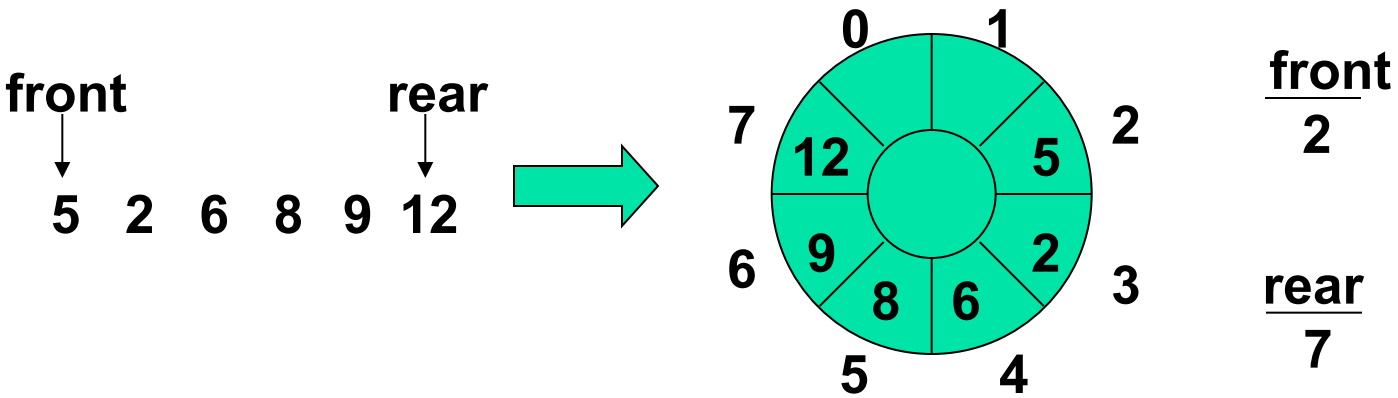
The problem here is that the rear index cannot move beyond the last element in the array.

Implementation using Circular Array

- Using a circular array
- When an element moves past the end of a circular array, it wraps around to the beginning, e.g.
  - OOOOO7963 → 4OOOO7963 (after Enqueue(4))
  - After Enqueue(4), the <u>rear index</u> moves from 3 to 4.

Queue using Array

- Basic idea is to picture the array as a *circular array*.

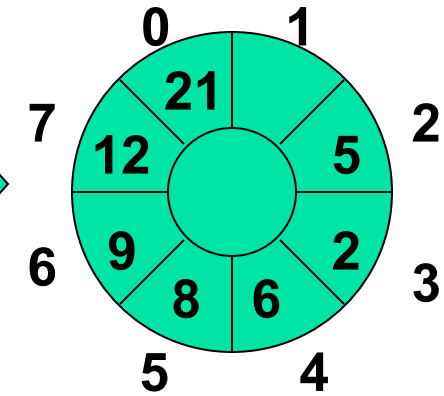


Queue using Array

enqueue(21)

front rear

5 2 6 8 9 12 21



front
2

size
8

rear
0

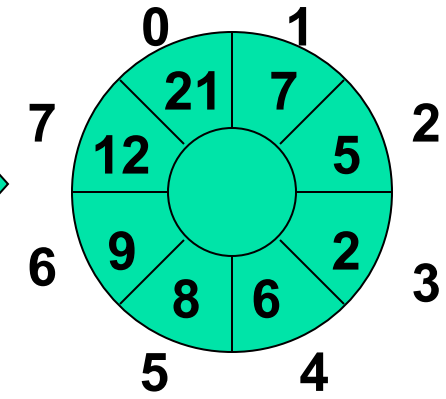
noElements
7

Queue using Array

enqueue(7)

front rear

5 2 6 8 9 12 21 7



front
2

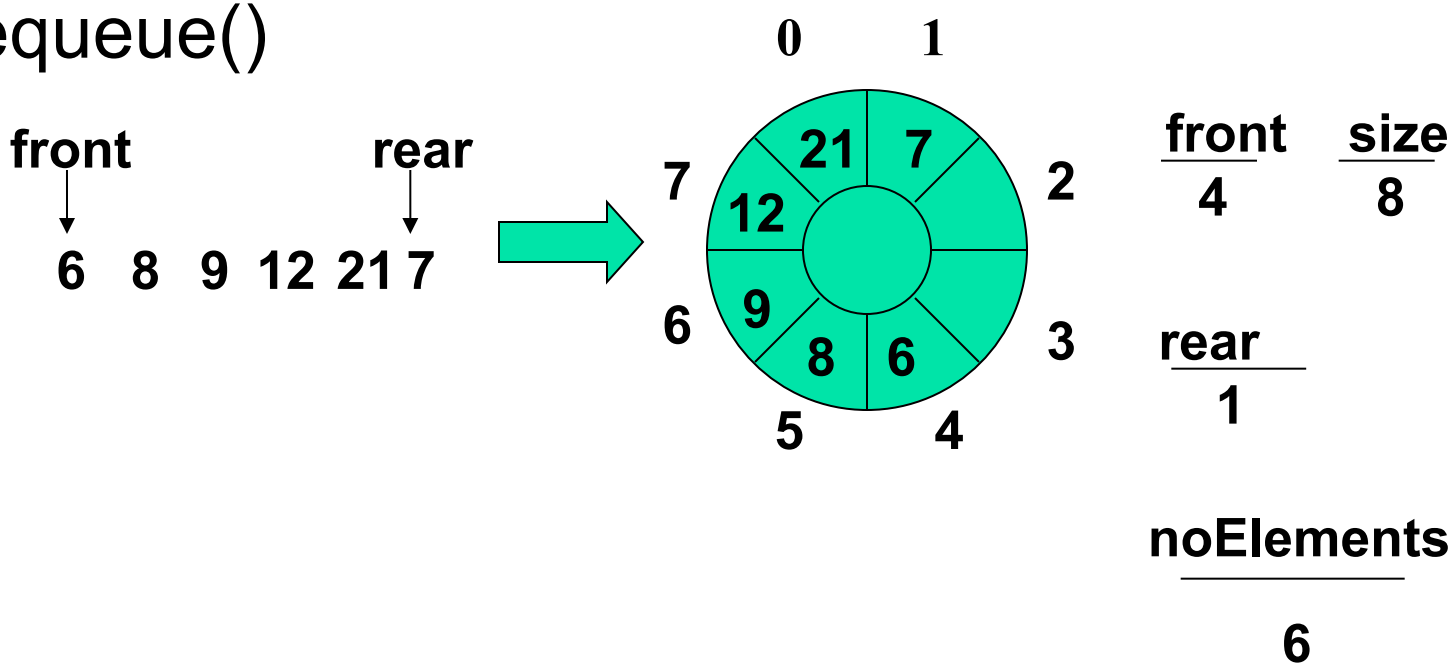
size
8

rear
1

noElements
8

Queue using Array

dequeue()



Queue Abstract Data Type

- **Queue( )** creates a new queue that is empty.
- **Enqueue(X)** adds a new item X at the rear of the queue.
- **Dequeue( )** remove the front element and return it.
- **Front( )** return front element without removing it.
- **isEmpty( )** tests to see whether the queue is empty.
- **isFull( )** tests to see whether the queue is Full.
- **size( )** returns the number of items on the Queue.
- **Destroy( )** deletes the content of the queue

Steps in Enqueue Operation

Step 1: Check the queue for overflow (full),

Step 2: if full insertion operation can't be performed produce overflow error & exit

Step 3: if queue is not full, increment rear pointer to point the next empty space.

Step 4: Add data element to the queue location, where the rear is pointing.

Step 5: Exit

Algorithm for Enqueue Operation

This algorithm will insert an element (item) on Queue (Q) and N=Size of Queue or No. of memory locations.

FRONT is used for Front pointer & REAR is used for rear Pointer

QINSERT (Q, FRONT,REAR,ITEM)

Step 1: if (REAR==N) //*Check for Overflow*

then write("Overflow") & exit;

Step 2: if (FRONT=-1 && REAR=-1) //Queue is empty

then FRONT=0; REAR=0;

else REAR=REAR+1; //*Increment Rear*

Step 3: Q[REAR]=newitem(value) //*Insert Value*

Step 4: Exit //*Finish*

Steps in dequeue Operation

- **Step 1** – Check if queue is empty.
- **Step 2** – If queue is empty, produce underflow error and exit.
- **Step 3** – If queue is not empty, access the data where **front** is pointing.
- **Step 4** – Increment **front** pointer to point to the next available data element.
- **Step 5** – Return success.

Algorithm for dequeue Operation

This algorithm will delete an element (item) on Queue (Q) and N=Size of Queue or No. of memory locations.

FRONT is used for Front pointer & REAR is used for rear Pointer

QDELETE (Q, FRONT,REAR,ITEM)

Step 1: if (FRONT==-1) //*Check for Underflow*

then write("Under flow") & exit;

Step 2: else Q[FRONT]=temp) //Queue is not empty

if (FRONT==REAR)

REAR=-1 && FRONT=-1) //*Set Pointers*

Step 3: **else** FRONT=FRONT+1;

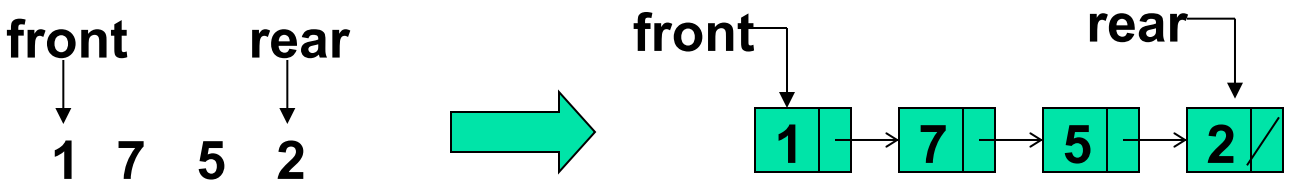
Step 4: Exit //*Finish*

Implementation of Queue in C++

```
#include<iostream.h>
#include<conio.h>
void QINSERT( );
void QDELETE( );
void display( );
int Q[5], FRONT=-1, REAR=-1, item;
void main( )
{
  int choice;
  clrscr( );
   cout<<"Enter your choice \n 1.Insert\n
              2. Delete  3. Display "<<endl;
    cin>>choice;
    switch(choice)
{  case 1: QINSERT( ); break;
case 2: QDELETE( ); break;
 case 3: display( );  break;
 default:   cout<<"Wrong Value";
}
getch( );
}
void QINSERT( )
{
 if (REAR= =4)
  { cout<<"\n Queue is Full";
break;}
If ( FRONT==-1&& REAR==-1)
{ FRONT ==0 && REAR == 0;}
Else REAR=REAR+1;
cout<<"Enter value to store";
cin>>Q[REAR];
}
 void deletion( )
{
if (FRONT = = 5)
 cout<<"Queue is
empty"<<endl;
 else Q[FRONT] = temp;
 FRONT= FRONT+1;
}
void display( )
{
for (int i=0; i<=top; i++)
cout<<Q[i];
}
```

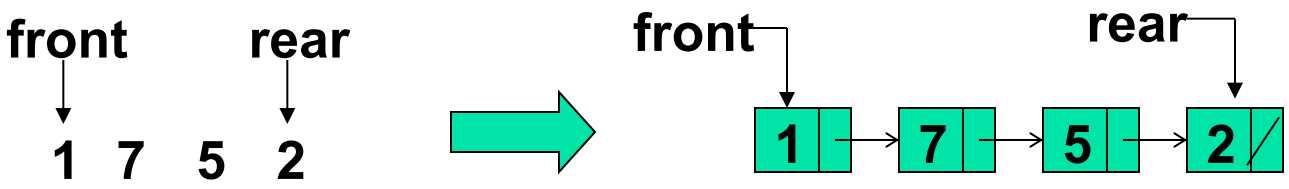
Implementing Queue

- Using linked List:

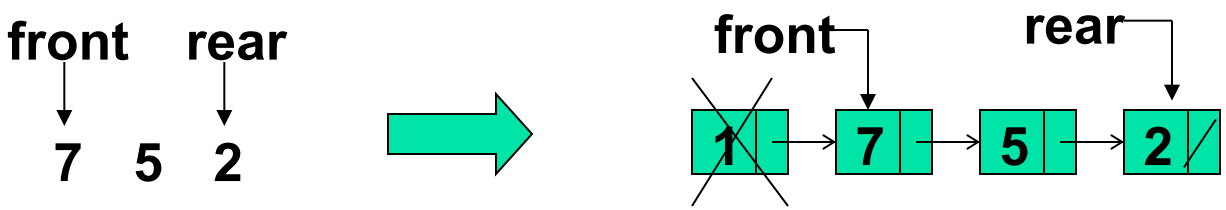


Implementing Queue

- Using linked List:

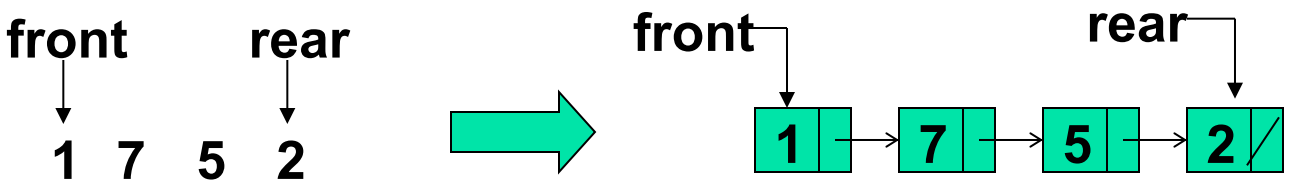


dequeue()



Implementing Queue

- Using linked List:

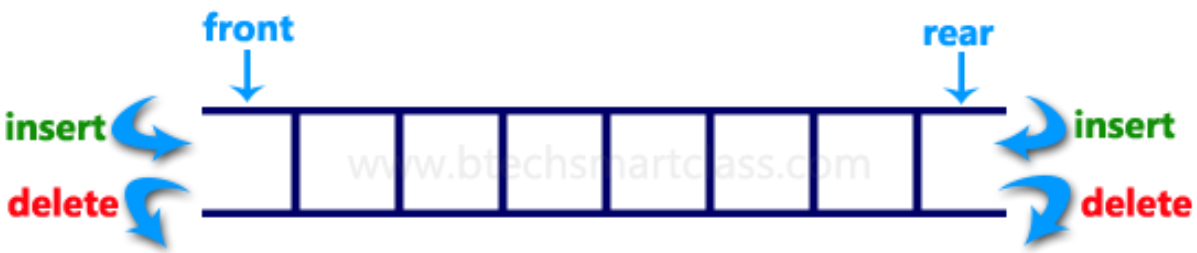


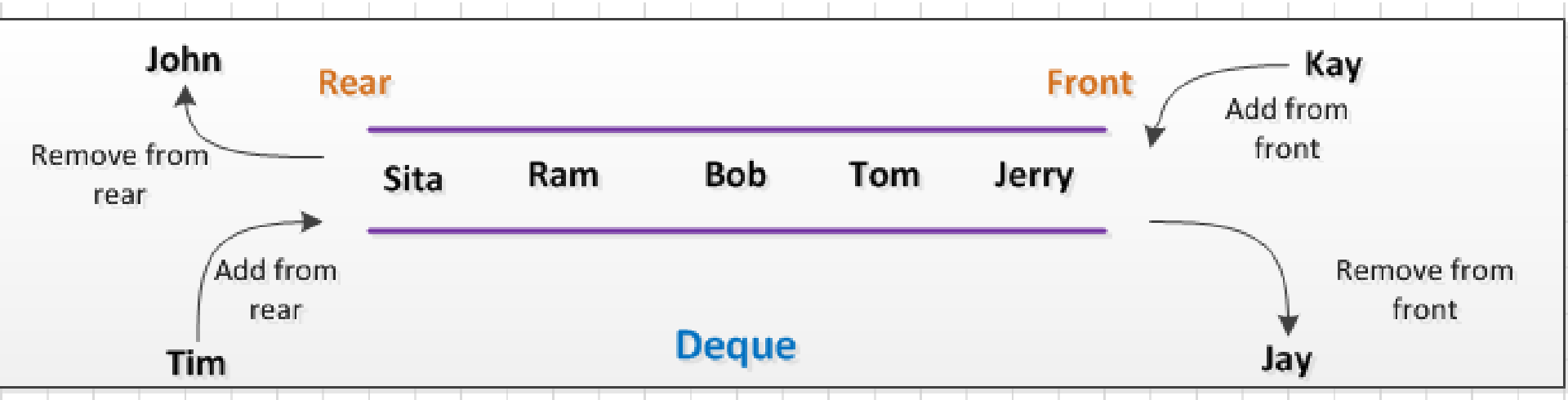
enqueue(9)

Deque (Double Ended Queue)

- A linear data structure of ordered homogeneous items
- It does not follow LIFO or FIFO
- It has two ends FRONT and REAR
- But we can and delete items from both sides but not from the middle
- It provides both the capability of Stack and Queue
- It’s a unique hybrid linear data structure
- Data in Deque is stored in memory using doubly link list

Deque


front
rear
insert
insert
delete
delete


John
Rear
Front
Kay
Add from front
Remove from rear
Sita
Ram
Bob
Tom
Jerry
Add from rear
Remove from front
Tim
Deque
Jay

Deque Abstract Data Type

- **Deque()** creates a new deque that is empty.
- **addFront(item)** adds a new item to the front of the deque.
- **addRear(item)** adds a new item to the rear of the deque.
- **removeFront()** removes the front item from the deque.
- **removeRear()** removes the rear item from the deque.
- **isEmpty()** tests to see whether the deque is empty.
- **size()** returns the number of items in the deque.